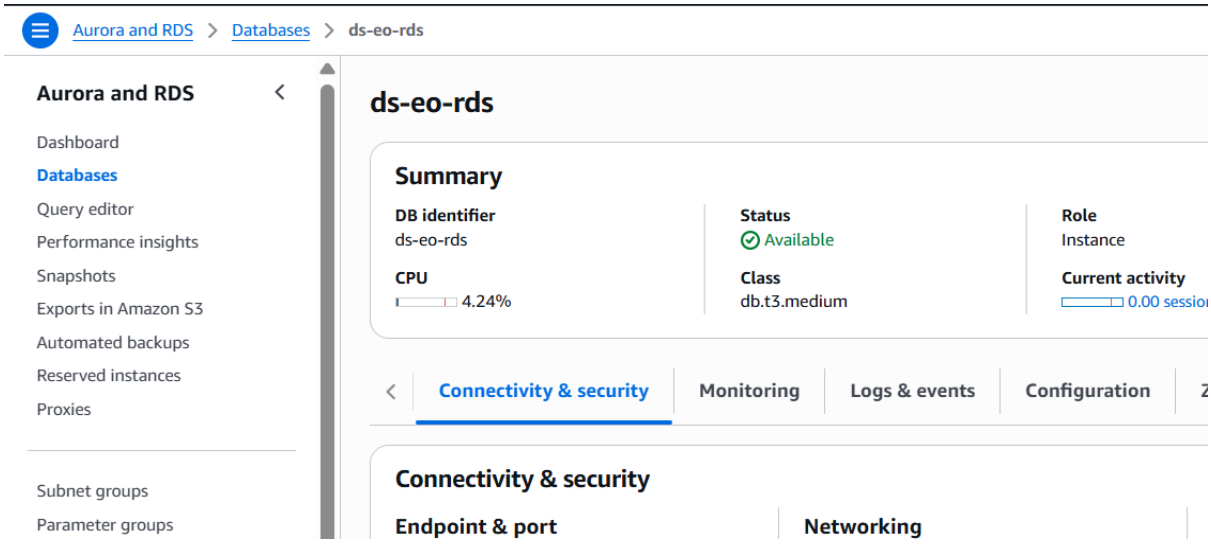
Plan of Action – RDS Database Isolation for EO & IS Applications

Current Setup:

One MySQL RDS instance (single endpoint) → being shared by all services of EO and ISOCS.



Inside that RDS:

Each microservice has its own database/schema (cart_service_dev, Coupon_Service_dev, is_gs_antenna_settings_dev, etc.).

The Parameter Store env files hold DB connection details (host, port, username, password, database).

Example:

agent_data_processing

central_gs_antenna_settings_dev

cart_service_dev

Coupon_Service_dev

So in short:

1 RDS → many schemas → used by ~15+ services per app (EO & IS) across dev/uat/prod.

Proposed Architecture

We will segregate RDS instances per application and environment:

IS

Dev/UAT → New db.t3.small (single-AZ) RDS MySQL instance

Prod → New db.t3.small (Multi-AZ HA) RDS MySQL instance

EO

Dev/UAT → New db.t3.small (single-AZ) RDS MySQL instance

Prod → New db.t3.small (Multi-AZ HA) RDS MySQL instance

This ensures environment and application isolation while maintaining cost efficiency.

That means after migration we'll have:

App	Env	DBs inside RDS (examples)
ISOCS	Dev/UAT	`isocs_bp_agent_data_processing`, `isocs_gs_entity_dev`, `isocs_uat_antenna`, …
ISOCS	Prod	`isocs_bp_agent_data_processing_prod`, `isocs_central_gs_antenna_settings_prod`, …
EO	Dev/UAT	`cart_service_dev`, `Coupon_Service_dev`, `order_service_uat`, …
EO	Prod	`cart_service_prod`, `Coupon_Service_prod`, `order_service_prod`, …

Pre-migration checklist

1.Inventory databases (schemas) per app & env:

login to current MySQL RDS

mysql -h <old-rds-endpoint> -P 33306 -u admin -p -e "SHOW DATABASES\G"

Save the list to a file and group by is_* and eo_* and by suffix _dev/_uat/_prod patterns.

2. Decide migration order (IS first — dev/uat -> prod -> EO dev/uat -> EO prod).

3. Backup: take a snapshot of the current RDS before operations

Migration Plan

Step 1: Snapshot Creation

Take a manual snapshot of the existing RDS instance as soon as the downtime starts.

This snapshot will contain all databases (EO + ISOCS + Dev/UAT/Prod).

Step 2: Restore Snapshot into New Instances

Restore the snapshot four times to create:

- ISOCS Dev/UAT RDS
- ISOCS Prod RDS (Multi-AZ enabled)
- EO Dev/UAT RDS
- EO Prod RDS (Multi-AZ enabled)

Step 3: Database Cleanup

For each new RDS instance:

ISOCS RDS → Drop EO-related databases.

EO RDS → Drop ISOCS-related databases.

Example cleanup command:

DROP DATABASE cart_service_dev;

DROP DATABASE coupon_service_dev;

Step 4: Application Reconfiguration

Update AWS Parameter Store with the new DB_HOST values per service.

Redeploy backend services via Jenkins pipelines, so containers connect to the new RDS endpoints.

Step 5: Validation

Dev/UAT environments validated by respective developer teams.

Production cutover scheduled with planned downtime, followed by post-migration validation.

Rollback Strategy:

- Keep current RDS running during migration.
- If validation fails, switch DB_HOST in Parameter Store back to old RDS and redeploy.
- No destructive action until all applications are validated.